

Labo 18 – Verslag JavaScript deel 6

opleidingsonderdeel **Web Development I**

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Tibo Dewanckele

docent: **Pim Debaere**

academiejaar **2025–2026**



Inhoud

Demo events.....	3
DOM Tree nodes.....	3
Oefeningen nodes	5
Opdracht 1	5
Opdracht 2	5
Opdracht 3	7
Demo event bubbling.....	9
Colorpicker uitbreiding	10

Demo events

blablabla 1

blablabla 2

blablabla 3

blablabla 4

blablabla 5

Klik op een paragraaf om diens tekst rood te maken

- 1) Ophalen van paragrafen met class text (zijn ze allemaal)
- 2) Aan alle paragrafen een **addEventListener** voor geklik toevoegen
- 3) Via **event.target** die specifieke paragraaf kunnen aanpassen
→ tekst naar rood

```
const setup = () => {
  let texts=document.querySelectorAll(".text");
  for (let i=0;i<texts.length;i++) {
    texts[i].addEventListener("click", klik);
  }
}

const klik = (event) => {
  event.target.style.color="red";
};

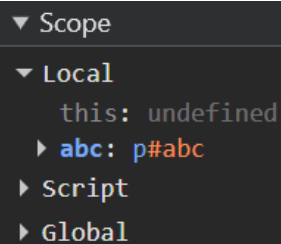
window.addEventListener("load", setup);
```

DOM Tree nodes

Volgende deel staat in geldige HTML.

```
<p id="abc">
  Hello World!
  <span>dit is een</span>
  tekst
</p>
```

Achter het refreshen van de pagina en het zetten van een breakpoint in javascript bij ophalen van element met id = abc.



```
▼ Scope
  ▼ Local
    this: undefined
    ▶ abc: p#abc
    ▶ Script
    ▶ Global
```

De childnodes zijn een tekst, span en nog een tekst. In de HTML is dit ook al zichtbaar.

```
▼ abc: p#abc
  ▶ attributeStyleMap: StylePr
  ▶ attributes: NamedNodeMap {
    baseURI: "http://localhost
    childElementCount: 1
  ▼ childNodes: NodeList(3)
    ▶ 0: text
    ▶ 1: span
    ▶ 2: text
    length: 3
  ▶ [[Prototype]]: NodeList
```

Als er verder op deze childnodes geklikt wordt kan je er nog veel over terugvinden zoals de inhoud ervan enzovoort. Hier zie je bij **data** dat de inhoud ervan in staat.

```
▼ 0: text
  baseURI: "http://localhost:63342/webdevelopment2/1
  ▶ childNodes: NodeList []
  data: "\n      Hello World!\n      "
```

De parentNode van abc is de body want het staat rechtsreeks in de body.

```
▶ parentNode: body
```

Om nu ook document te kunnen bekijken moeten we bij **Watch** op het plusje drukken.

```
▼ Watch + ↺
```

Dan typ je document en kan je document volgen.

```
▼ childNodes: NodeList(2)
  ▶ 0: <!DOCTYPE html>
  ▶ 1: html
  length: 2
  ▶ [[Prototype]]: NodeList
```

Document heeft 2 childnodes (DOCTYPE en html, deze hebben geen type)

Document heeft geen parentNode want dit is het bovenste deel in de DOM-tree.

Oefeningen nodes

Opdracht 1

Goed gedaan!

Volgende regel staat in HTML

```
<p>Verander mij!</p>
```

We willen de inhoud veranderen naar “Goed gedaan!”

Als we **querySelectorAll** gebruiken dan kunnen we alle paragrafen ophalen en via [0] halen we de 1^{ste} op.

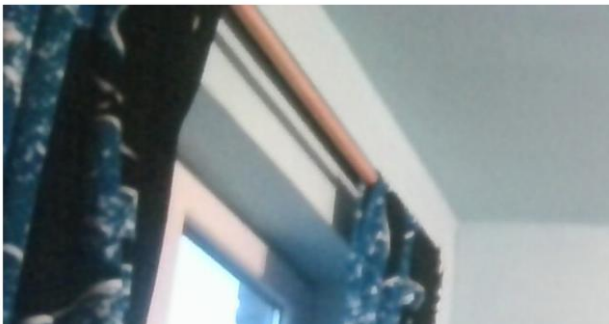
Via **textContent** kunnen we dan de inhoud aanpassen.

```
let pElement = document.querySelectorAll("p")[0];  
pElement.textContent = "Goed gedaan!"
```

Opdracht 2

About Me

- Nickname:
- Favorieten:
- Thuisstad:



We kunnen wat er moet gedaan worden in javascript verdelen over 3 methoden.

```
Const setup = () => {  
  createStyleElement()  
  geefLiElementenListItemKlasse();  
  maakImgElement();  
}
```

We halen eerst de head van de HTML op want hierin willen we een style blok toevoegen.

Dan gaan we een style blok creëren via **createElement**.

Dan creëren we een tekst om in de style blok te steken via **createTextNode**.

Via **appendChild** kunnen we nu de style blok toevoegen aan head en dan nog de tekst toevoegen aan de style blok.

```
const createStyleElement = () => {
  let head = document.querySelector("head");

  let style = document.createElement("style");
  let text = document.createTextNode(".listItem{color:red}");

  head.appendChild(style);
  style.appendChild(text);
}
```

We willen nu de class dat we hebben gebruikt in de style blok aan alle li elementen toekennen. We halen alle li elementen op via **querySelectorAll**.

Via een for-lus kunnen we dit nu toekennen aan elk li element via **setAttribute**.

We geven aan dat het een class is en dat de waarde ervan listItem is.

```
const geefLiElementenListItemKlasse = () => {
  let liElementen = document.querySelectorAll("li");
  for(let i=0; i<liElementen.length; i++) {
    liElementen[i].setAttribute("class","listItem");
  }
}
```

Nu willen we nog een afbeelding toevoegen.

We halen de body op omdat hierin de image moet staan.

Dan creëren we een img element via **createElement**.

Dan geven we het pad mee via **setAttribute**.

We voegen dit dan toe aan de body.

```
const maakImgElement = () => {
  let body = document.querySelector("body");
  let img = document.createElement("img");
  img.setAttribute("src","img/Tibo_Dewanckele.jpg.jpg");

  body.appendChild(img);
}
```

Opdracht 3

```
<!DOCTYPE html>
<html lang="nl">
<head>
  <style>
    #myDIV {
      border: 1px solid black;
      margin-bottom: 10px;
    }
  </style>
</head>
<body>
<p>
  Klik op een knop om een p-element aan te maken.
</p>
<div id="myDIV">
  Dit is een div-element
</div>
<button>Toevoegen p element</button>
<script src="scripts/script.js"></script>
</body>
</html>
```

Dit is de HTML voor deze opdracht en creert volgende pagina.

Klik op een knop om een p-element aan te maken.

Dit is een div-element

Toevoegen p element

We willen dat als we op de knop drukken dat er een p element binnen het kader wordt toegevoegd. We controleren dus eerst of er op gedrukt wordt.

```
const setup = () => {
  let button = document.querySelector("button");
  button.addEventListener("click", voegPToeAanDiv)
}
```

We halen dus eerst de div op en maken dan een p element aan, dan maken we ook de tekst ervoor aan.

Dit voegen we dan aan de div toe en dan ook nog eens de tekst aan het p element.

```
const voegPToeAanDiv = () => {  
  let div = document.querySelector("div");  
  let p = document.createElement("p");  
  let txt = document.createTextNode("p");  
  div.appendChild(p);  
  p.append(txt);  
}
```

Klik op een knop om een p-element aan te maken.

Dit is een div-element

p

p

p

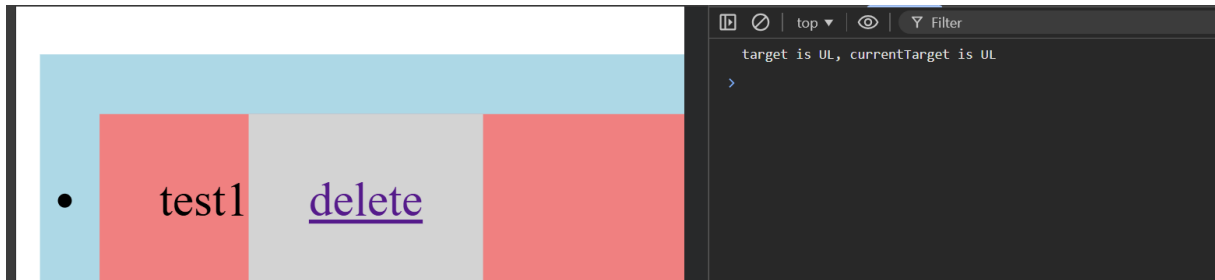
p

p

Toevoegen p element

Demo event bubbling

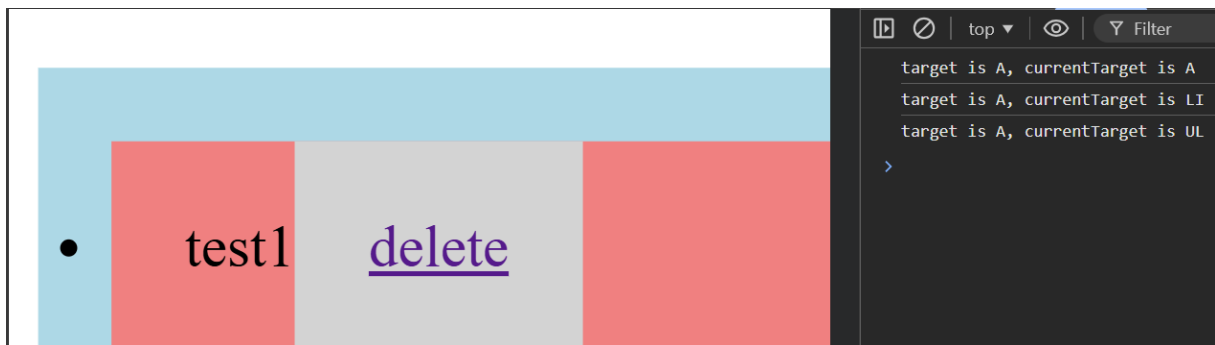
Als je op blauw (**UL**) drukt



Als je op rood (**LI**) drukt



Als je op grijs (**A**) drukt



Conclusie: **target** = hetgeen waarop dat je rechtsreeks klikt

currentTarget = hetgeen dat op een bepaald moment een methode wordt voor uitgevoerd en iets kan meerdere keren uitgevoerd worden omdat het achter elkaar zit en dus zogezegd aangeklikt wordt.

Colorpicker uitbreiding

The image shows a color picker interface. It features three horizontal sliders for Red, Green, and Blue. The Red slider is set to 255, while Green and Blue are at 0. To the right of the sliders is a large red square representing the current color. A 'Save' button is located to the right of the color square. Below the main interface are four color swatches: a white swatch, a red swatch, a magenta swatch, and a blue swatch. Each swatch has a small 'X' button in its top-left corner.

We halen dus nog steeds de slider op voor de waardes en het vierkant up te daten.
Maar nu hebben we nog de knop om een kopie ervan te maken.

```
const setup = () => {  
  let sliders = document.getElementsByClassName("slider");  
  
  for (let i = 0; i < sliders.length; i++) {  
    sliders[i].addEventListener("input", update);  
    sliders[i].addEventListener("change", update);  
  }  
  
  let button = document.querySelector("button");  
  button.addEventListener("click", kopieSwatchMaken);  
  
  update();  
};
```

We updaten dus de kleur van het vierkant maar via **setAttribute** bewaren we de kleuren zodat we dit kunnen toekennen aan de kopie als we op de knop drukken.

```
const update = () => {
  let sliders = document.getElementsByClassName("slider");
  let values = document.getElementsByTagName("span");

  let red = sliders[0].value;
  let green = sliders[1].value;
  let blue = sliders[2].value;

  values[0].textContent = red;
  values[1].textContent = green;
  values[2].textContent = blue;

  let colorDemo = document.getElementsByClassName('colorDemo');
  colorDemo[0].style.backgroundColor = `rgb(${red}, ${green}, ${blue})`;
  colorDemo[0].setAttribute("rood", red);
  colorDemo[0].setAttribute("groen", green);
  colorDemo[0].setAttribute("blauw", blue);
}
```

Om een kopie te maken halen we eerst het originele vierkant op. We creëren een kopie en kennen er de kleuren aan toe dat we hadden bewaard vooraf (**getAttribute**).

Dan creëren we een knop X voor op de kopie om hem te kunnen verwijderen.

We kennen er CSS aan toe om te zorgen dat het goed staat.

We kennen opnieuw de attributes toe voor de kleuren.

```
const kopieSwatchMaken = () => {
  let swatch = document.querySelector(".colorDemo");

  let kopie = document.createElement("div");
  let rood = Number(swatch.getAttribute("rood"));
  let groen = Number(swatch.getAttribute("groen"));
  let blauw = Number(swatch.getAttribute("blauw"));

  let button = document.createElement("button");
  let txt = document.createTextNode("X");

  let divKopies = document.querySelector("#kopie");
  divKopies.appendChild(kopie);
  kopie.style.backgroundColor = `rgb(${rood}, ${groen}, ${blauw})`;
  kopie.style.height = "100px";
  kopie.style.width = "100px";
  kopie.style.border = "1px solid black";
  kopie.style.margin = "5px";
  kopie.style.display = "inline-block";
}
```

```

    kopie.setAttribute("rood", String(rood));
    kopie.setAttribute("groen", String(groen));
    kopie.setAttribute("blauw", String(blauw));

    kopie.appendChild(button);
    button.append(txt);

    button.addEventListener("click", (event) => {
        event.stopPropagation();
        kopie.remove();
    });

    kopie.addEventListener("click", zetSlidersTerug);
}

```

We voegen een **addEventListener** toe aan beide want er zijn acties aan gekoppeld.

Bij de knop gaan we het gewoon verwijderen en om te zorgen dat de kleur niet veranderd van het originele vierkant wat gewenst is bij het klikken van de kopie, maken we gebruik van **stopPropagation** dit zorgt ervoor dat hij niet doorgaat naar de parent van de button.

Hier halen we de kopie op waarop dat we juist hebben gedrukt.

We halen de kleuren op die in de attributes zaten en dan kennen we deze toe aan de sliders. Dan roepen we de methode **update** op die we eerder hadden geschreven om het vierkant te updaten.

```

const zetSlidersTerug = (event) => {
    const kopie = event.currentTarget;

    const rood = kopie.getAttribute("rood");
    const groen = kopie.getAttribute("groen");
    const blauw = kopie.getAttribute("blauw");

    const sliders = document.getElementsByClassName("slider");

    sliders[0].value = rood;
    sliders[1].value = groen;
    sliders[2].value = blauw;

    update();
}

```